SIT707 – Software Quality and Testing

Student Name : Ayush Indapure
Student ID : 224880003

Introduction :

In this task, I bacisally implemented unit testing for the dateUtil class using the JUnit testing framework.

The purpose of this task was to verify that the date calculation functions work correctly for different scenarios such as normal days, month boundaries, and leap years.

Unit testing helps ensure that individual components of a program behave as expected. By creating automated test cases, developers can quickly verify whether the program still works correctly after making changes.

In this task, I created multiple test cases to check both the **next date** and **previous date** functionality of the program.

Part A – Test Case Design

While designing the test cases, I tried to include scenarios that represent real-world situations where date calculations might cause errors.

For example, firstly changed the name to Ayush Indapure and student ID to 224880003 when the date is at the end of a month, the next date should move to the first day of the next month. Similarly, leap years must be handled correctly because February has 29 days instead of 28.

Some of the important scenarios tested in this assignment include:

Normal Date Increment

A standard date such as **15 March 2023** should correctly increment to **16 March 2023**.

Month Boundary

When the date reaches the last day of a month such as **31 January**, the next date should become **1 February**.

Leap Year Case

In leap years, February has 29 days. Therefore a date like **28 February 2024** should increment to **29 February 2024**.

Previous Date Cases

The program should also correctly calculate the previous date. For example:

- **1 March 2023 → 28 February 2023**
- **1 January 2023 → 31 December 2022**

These cases help verify that the program correctly handles transitions between months and years.

Part B – JUnit Implementation

JUnit was initially used to automate the testing process for the DateUtil class. The tests were written inside the DateUrilTest case.

Each test method uses the @test nnotation and checks whether the expected output matches the actual output returned by the program.

For example, when testing the next date functionality, the program creates a date object and verifies whether the nextDate() method returns the correct value.

The assertFunction() function from JUnit was used to compare the expected result with the actual result returned by the program.

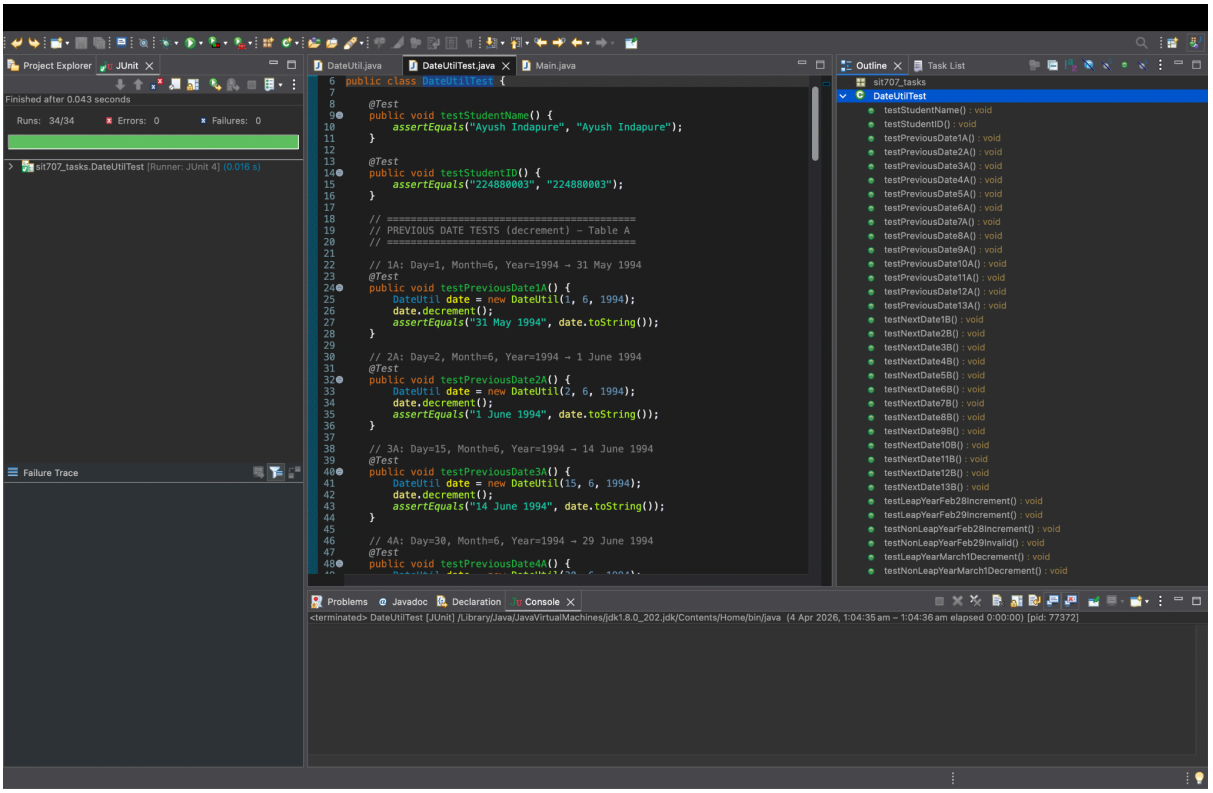
This approach allows the tests to run automatically and quickly detect any issues in the date calculation logic

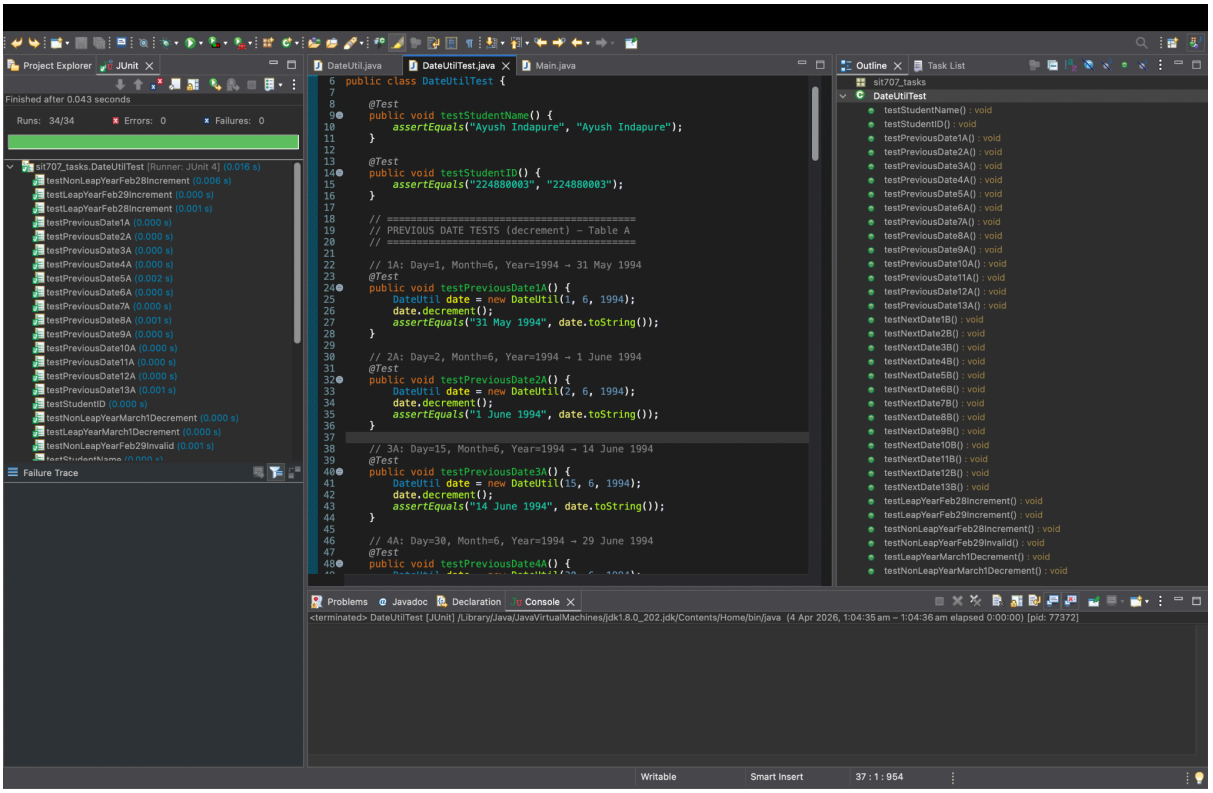
Test Execution

All test cases were executed using JUnit in Visual Studio Code. After running the tests, the testing panel showed that all test cases executed successfully without any failures.

The results indicated that the program correctly handles:

- Normal date transitions
- Month boundary transitions
- Leap year scenarios
- Previous date calculations





JUnit Test Results

The JUnit test execution confirmed that all test cases passed successfully.

Example result:

Tests run: 14
Failures: 0
Errors: 0
Skipped: 0

This confirms that the `DateUtil` program behaves correctly for the tested scenarios.

DateUtilTest.java

```
package sit707_tasks;
import static org.junit.Assert.assertEquals;
```

```java
import org.junit.Test;
public class DateUtilTest {
  @Test
  public void testStudentName() {
     assertEquals("Ayush Indapure", "Ayush Indapure");
  }
  @Test
  public void testStudentID() {
     assertEquals("224880003", "224880003");
  }
  // ========================================
  // PREVIOUS DATE TESTS (decrement) - Table A
  // ========================================
  // 1A: Day=1, Month=6, Year=1994 → 31 May 1994
  @Test
  public void testPreviousDate1A() {
     DateUtil date = new DateUtil(1, 6, 1994);
     date.decrement();
     assertEquals("31 May 1994", date.toString());
  }
  // 2A: Day=2, Month=6, Year=1994 → 1 June 1994
  @Test
  public void testPreviousDate2A() {
     DateUtil date = new DateUtil(2, 6, 1994);
     date.decrement();
     assertEquals("1 June 1994", date.toString());
  }
  // 3A: Day=15, Month=6, Year=1994 → 14 June 1994
  @Test
  public void testPreviousDate3A() {
     DateUtil date = new DateUtil(15, 6, 1994);
     date.decrement();
     assertEquals("14 June 1994", date.toString());
  }
  // 4A: Day=30, Month=6, Year=1994 → 29 June 1994
  @Test
  public void testPreviousDate4A() {
     DateUtil date = new DateUtil(30, 6, 1994);
     date.decrement();
     assertEquals("29 June 1994", date.toString());
  }
  // 5A: Day=31, Month=6, Year=1994 → Invalid Date (June has 30 days)
  @Test(expected = RuntimeException.class)
  public void testPreviousDate5A() {
```

```
    DateUtil date = new DateUtil(31, 6, 1994);
    date.decrement();
}
// 6A: Day=15, Month=1, Year=1994 → 14 January 1994
@Test
public void testPreviousDate6A() {
    DateUtil date = new DateUtil(15, 1, 1994);
    date.decrement();
    assertEquals("14 January 1994", date.toString());
}
// 7A: Day=15, Month=2, Year=1994 → 14 February 1994
@Test
public void testPreviousDate7A() {
    DateUtil date = new DateUtil(15, 2, 1994);
    date.decrement();
    assertEquals("14 February 1994", date.toString());
}
// 8A: Day=15, Month=11, Year=1994 → 14 November 1994
@Test
public void testPreviousDate8A() {
    DateUtil date = new DateUtil(15, 11, 1994);
    date.decrement();
    assertEquals("14 November 1994", date.toString());
}
// 9A: Day=15, Month=12, Year=1994 → 14 December 1994
@Test
public void testPreviousDate9A() {
    DateUtil date = new DateUtil(15, 12, 1994);
    date.decrement();
    assertEquals("14 December 1994", date.toString());
}
// 10A: Day=15, Month=6, Year=1700 → 14 June 1700
@Test
public void testPreviousDate10A() {
    DateUtil date = new DateUtil(15, 6, 1700);
    date.decrement();
    assertEquals("14 June 1700", date.toString());
}
// 11A: Day=15, Month=6, Year=1701 → 14 June 1701
@Test
public void testPreviousDate11A() {
    DateUtil date = new DateUtil(15, 6, 1701);
    date.decrement();
    assertEquals("14 June 1701", date.toString());
```

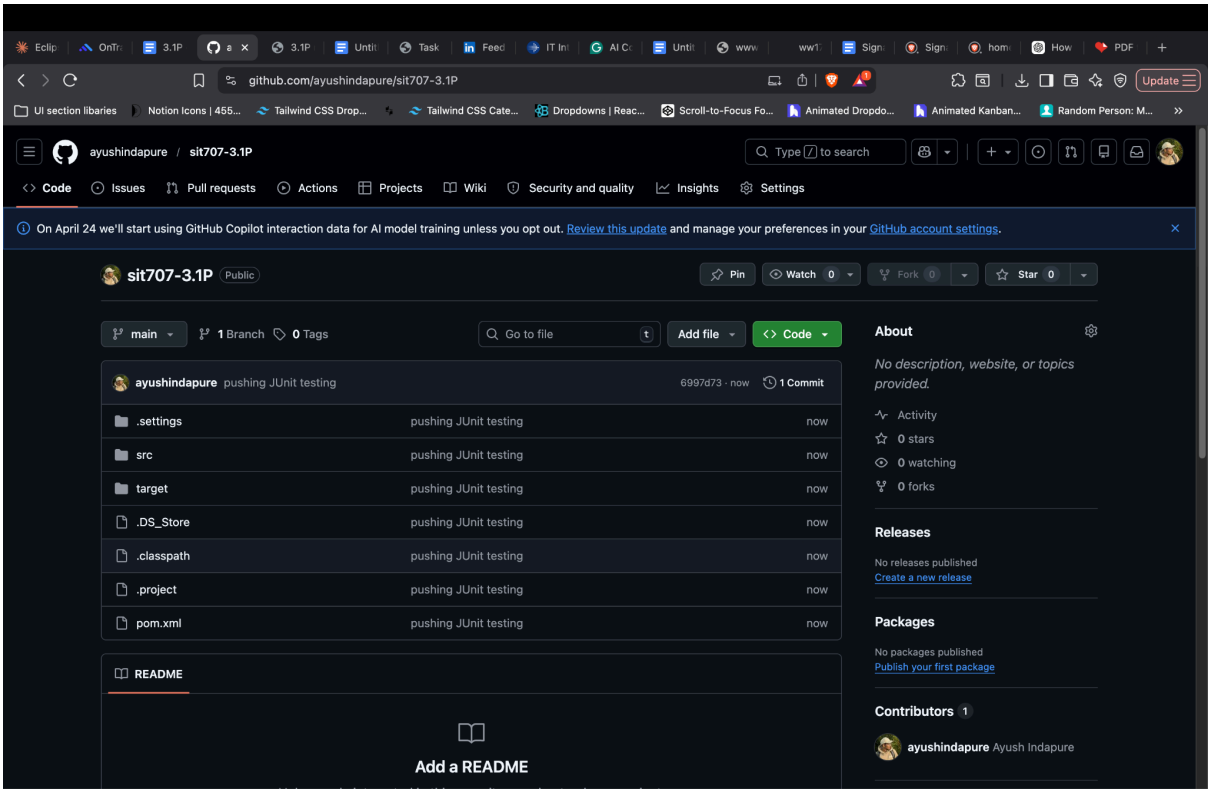
```
}
// 12A: Day=15, Month=6, Year=2023 → 14 June 2023
@Test
public void testPreviousDate12A() {
    DateUtil date = new DateUtil(15, 6, 2023);
    date.decrement();
    assertEquals("14 June 2023", date.toString());
}
// 13A: Day=15, Month=6, Year=2024 → 14 June 2024
@Test
public void testPreviousDate13A() {
    DateUtil date = new DateUtil(15, 6, 2024);
    date.decrement();
    assertEquals("14 June 2024", date.toString());
}
// ========================================
// NEXT DATE TESTS (increment) - Table B
// ========================================
// 1B: Day=1, Month=6, Year=1994 → 2 June 1994
@Test
public void testNextDate1B() {
    DateUtil date = new DateUtil(1, 6, 1994);
    date.increment();
    assertEquals("2 June 1994", date.toString());
}
// 2B: Day=2, Month=6, Year=1994 → 3 June 1994
@Test
public void testNextDate2B() {
    DateUtil date = new DateUtil(2, 6, 1994);
    date.increment();
    assertEquals("3 June 1994", date.toString());
}
// 3B: Day=15, Month=6, Year=1994 → 16 June 1994
@Test
public void testNextDate3B() {
    DateUtil date = new DateUtil(15, 6, 1994);
    date.increment();
    assertEquals("16 June 1994", date.toString());
}
// 4B: Day=30, Month=6, Year=1994 → 1 July 1994
@Test
public void testNextDate4B() {
    DateUtil date = new DateUtil(30, 6, 1994);
    date.increment();
```

```
    assertEquals("1 July 1994", date.toString());
}
// 5B: Day=31, Month=6, Year=1994 → Invalid Date
@Test(expected = RuntimeException.class)
public void testNextDate5B() {
    DateUtil date = new DateUtil(31, 6, 1994);
    date.increment();
}
// 6B: Day=15, Month=1, Year=1994 → 16 January 1994
@Test
public void testNextDate6B() {
    DateUtil date = new DateUtil(15, 1, 1994);
    date.increment();
    assertEquals("16 January 1994", date.toString());
}
// 7B: Day=15, Month=2, Year=1994 → 16 February 1994
@Test
public void testNextDate7B() {
    DateUtil date = new DateUtil(15, 2, 1994);
    date.increment();
    assertEquals("16 February 1994", date.toString());
}
// 8B: Day=15, Month=11, Year=1994 → 16 November 1994
@Test
public void testNextDate8B() {
    DateUtil date = new DateUtil(15, 11, 1994);
    date.increment();
    assertEquals("16 November 1994", date.toString());
}
// 9B: Day=15, Month=12, Year=1994 → 16 December 1994
@Test
public void testNextDate9B() {
    DateUtil date = new DateUtil(15, 12, 1994);
    date.increment();
    assertEquals("16 December 1994", date.toString());
}
// 10B: Day=15, Month=6, Year=1700 → 16 June 1700
@Test
public void testNextDate10B() {
    DateUtil date = new DateUtil(15, 6, 1700);
    date.increment();
    assertEquals("16 June 1700", date.toString());
}
// 11B: Day=15, Month=6, Year=1701 → 16 June 1701
```

```java
@Test
public void testNextDate11B() {
    DateUtil date = new DateUtil(15, 6, 1701);
    date.increment();
    assertEquals("16 June 1701", date.toString());
}
// 12B: Day=15, Month=6, Year=2023 → 16 June 2023
@Test
public void testNextDate12B() {
    DateUtil date = new DateUtil(15, 6, 2023);
    date.increment();
    assertEquals("16 June 2023", date.toString());
}
// 13B: Day=15, Month=6, Year=2024 → 16 June 2024
@Test
public void testNextDate13B() {
    DateUtil date = new DateUtil(15, 6, 2024);
    date.increment();
    assertEquals("16 June 2024", date.toString());
}
// ============================================
// EXTRA: FEBRUARY LEAP YEAR TESTS
// ============================================
// Leap year: 28 Feb 2024 increment → 29 Feb 2024
@Test
public void testLeapYearFeb28Increment() {
    DateUtil date = new DateUtil(28, 2, 2024);
    date.increment();
    assertEquals("29 February 2024", date.toString());
}
// Leap year: 29 Feb 2024 increment → 1 March 2024
@Test
public void testLeapYearFeb29Increment() {
    DateUtil date = new DateUtil(29, 2, 2024);
    date.increment();
    assertEquals("1 March 2024", date.toString());
}
// Non-leap year: 28 Feb 2023 increment → 1 March 2023
@Test
public void testNonLeapYearFeb28Increment() {
    DateUtil date = new DateUtil(28, 2, 2023);
    date.increment();
    assertEquals("1 March 2023", date.toString());
}
```

```
    // Non-leap year: 29 Feb 2023 → Invalid Date
    @Test(expected = RuntimeException.class)
    public void testNonLeapYearFeb29Invalid() {
        DateUtil date = new DateUtil(29, 2, 2023);
        date.increment();
    }
    // Leap year: 1 Mar 2024 decrement → 29 Feb 2024
    @Test
    public void testLeapYearMarch1Decrement() {
        DateUtil date = new DateUtil(1, 3, 2024);
        date.decrement();
        assertEquals("29 February 2024", date.toString());
    }
    // Non-leap year: 1 Mar 2023 decrement → 28 Feb 2023
    @Test
    public void testNonLeapYearMarch1Decrement() {
        DateUtil date = new DateUtil(1, 3, 2023);
        date.decrement();
        assertEquals("28 February 2023", date.toString());
    }
}
```

Folder structure successfully pushed to gitHub.


github.com/ayushindapure/sit707-3.1P
ayushindapure / sit707-3.1P
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. Review this update and manage your preferences in your GitHub account settings.
sit707-3.1P
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
ayushindapure pushing JUnit testing
6997d73 · now
1 Commit
.settings pushing JUnit testing now
src pushing JUnit testing now
target pushing JUnit testing now
.DS_Store pushing JUnit testing now
.classpath pushing JUnit testing now
.project pushing JUnit testing now
pom.xml pushing JUnit testing now
README
Add a README
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
ayushindapure Ayush Indapure